claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962\agent-a66ebffea3c336406.jsonl`
- SHA-256: `3065c9092c10ff827903a7df753b6d0274e146b08eaaf403a021115bb735133d`
- Source modified: `2026-06-07T08:17:12+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_49729ae7-962`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T08:15:38.734Z)

You are a tough evaluator of how DEBUGGABLE an observability report is for a README-only reader.
Scenario: indexer_fps_fallback (tool: indexer)
GROUND TRUTH (what actually went wrong): fps was not probed from the file (a fallback default was used), so motion thresholds and frame->second timing may be miscalibrated. Confirm the real fps with ffprobe and ensure validation ran.
The report is expected to point the reader toward FAQ: "Q5".

THE FRESH READER ANSWERED:
- diagnosis: The FPS (frames-per-second) value was not probed from the video file and instead fell back to a default hardcoded value. This means all motion thresholds, frame-to-second timing conversions, and velocity calculations in the segmentation pipeline are using incorrect baseline assumptions. The report shows FPS | ? (fallback) in the Video section and the warning fps_fallback_used explicitly states fps was assumed (a fallback default), not probed from the file. Motion thresholds and frame->second timing may be miscalibrated.
- fix: 1. Run ffprobe directly on the video to extract the actual FPS: ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of default=noprint_wrappers=1:nokey=1:noesc=1 /videos/match.mp4. 2. If ffprobe returns empty or an error, the file likely uses HEVC/H.265 encoding (common in GoPro footage) where standard FFmpeg frame-skipping fails ? see README Q5. 3. Once the actual FPS is confirmed, re-run the indexing pipeline so that FPS is properly probed from the file during validation. 4. Alternatively, if FFprobe cannot extract FPS from this file format, manually set the correct FPS in config.json under validation.min_fps based on the confirmed value, then re-run.
- faq_section used: README#Q5 (Why did the video fail with "No keyframes / packet timestamps could be extracted"?)
- could_self_debug: true
- confusing_parts: ["Missing fallback value ? The report shows FPS | ? (fallback) but never states which numeric fallback was actually used (30? 60?). Showing the actual fallback value (e.g., FPS | 30 (fallback)) would immediately help validate if the assumption is wildly wrong.","Validation vs. FPS probing linkage ? The report shows validation passed (checks_passed: 1/1), but doesn't clarify whether FPS probing is part of validation or a separate step. If it's part of validation, why did validation pass but FPS still fall back?","No error context ? The report doesn't include the underlying error from FFprobe/FFmpeg (e.g., FFprobe not in PATH, stream has no r_frame_rate field, HEVC keyframe parsing failed). Including that would pinpoint the root cause immediately.","Q5 reference mismatch ? Q5 is titled about keyframes, but the actual fix for FPS fallback might be a different problem entirely. The warning message could benefit from citing Q1 (ffprobe not found) as an alternative possibility."]

Also read the actual report for context:
C:\Users\avidu\AppData\Local\Temp\obseval\reports\indexer_fps_fallback.report.md

Judge strictly: did the reader land on the correct ROOT CAUSE (cause_correct)? Did they find/consult the expected FAQ (faq_correct)? How large is the remaining gap (none/minor/major)? Then propose ONE concrete, specific refinement to the REPORT wording, the spec rule message, or the FAQ that would have made this scenario unambiguous to self-debug ? or "none" if it is

already excellent. Don't be generous; if anything was confusing, propose a fix.

2. assistant (2026-06-07T08:15:41.446Z)

I'll read the actual report to evaluate the reader's diagnosis against ground truth.

3. user (2026-06-07T08:15:41.832Z)

```
1       # Badminton Highlight Indexer ? Video Report
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Tool | badminton-highlight-indexer |
8       | Video id | md5d (file_md5) |
9       | Video path | /videos/match.mp4 |
10      | Generated | 2026-06-07T08:10:50Z |
11      | Schema | 1.0 |
12
13      ## Video
14
15      | Field | Value |
16      | --- | --- |
17      | FPS | ? (fallback) |
18
19      ## Processing stages
20
21      **validation** ? `ok` (0.0s)
22        - checks_passed: 1
23        - checks_total: 1
24
25      **segmentation** ? `ok` (0.0s)
26        - candidate_windows: 1
27        - segment_count: 1
28        - total_play_s: 5 s
29        - _segmenter_requested_: wasb_hybrid
30        - _segmenter_actual_: wasb_hybrid
31        - _config_default_: wasb_hybrid
32        - _segmenter_explicitly_requested_: True
33        - _matches_config_default_: True
34        - _was_reingest_: False
35        - _detector_: wasb-hybrid
36
37      **ai_handoff** ? `ok` (0.0s)
38        - confirmed_segments: 1
39        - _ai_based_: True
40        - _provider_: gemini
41        - _model_: None
42        - _api_key_present_: True
43
44      ## Findings & warnings
45
46      - [WARN] **fps_fallback_used** ? fps was assumed (a fallback default), not probed from
the file. Motion thresholds and frame->second timing may be miscalibrated. Confirm the real fps
with `ffprobe` and ensure validation ran (it probes the file).
47        ? see **README#Q5**
48        ? evidence: video_meta.fps_source=fallback
49
50      ## Verdict
51
52      **PASS (with warnings)**
53
```

4. assistant (2026-06-07T08:15:44.566Z)

Now let me check the README FAQ to verify what Q5 actually covers versus the FPS fallback scenario.

5. user (2026-06-07T08:15:44.684Z)

.pytest_cache\README.md
README.md

6. user (2026-06-07T08:15:48.831Z)

38:- **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
82:Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b -S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed.
The `fetch` command (re)downloads videos for entries already in the DB and updates `local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation status (no clean slate needed):
105:[Omitted long matching line]

7. assistant (2026-06-07T08:15:52.732Z)

The README in this repo (sports-data-collector) doesn't have a Q5 FAQ. The report is for the badminton-highlight-indexer tool. Let me find the actual FAQ that Q5 refers to.

8. user (2026-06-07T08:15:53.990Z)

No matches found

9. user (2026-06-07T08:16:00.256Z)

Found 13 files
docs\INGESTION_PIPELINE.md
src\cli\curate.py
src\reporting__init__.py
src\reporting\adapter.py
src\reporting\spec.yaml
tests\test_reporting.py
src\parsers\cvat\rally_cvat.py
tests\test_cvat_parser.py
docs\annotation\ANNOTATION_GUIDE.md
src\core\validator.py
src\core\config.py
tests\test_validator.py
src\core\downloader.py

10. assistant (2026-06-07T08:16:03.750Z)

The report is generated by this repo's reporting module. Let me look at the spec.yaml which likely defines the warning message, FAQ references, and the FAQ content itself.

11. user (2026-06-07T08:16:03.979Z)

```
1       # Observability report spec for the sports-data-collector.
2       # The report ENGINE lives in the shared `obsreport` library; this is the
3       # collector-specific layout + a couple of rules. Most delivery findings come
4       # straight from rally_cvat's Issue list (mapped 1:1 to flags by the adapter);
5       # the rules below cover signals that aren't delivery "issues".
6       name: collector
7       title: Sports Data Collector ? Delivery / Ingestion Report
8
9       # Customer here = the annotation vendor / data team receiving delivery feedback.
10      customer_verdict:
11        pass: "This delivery passed all checks and is ready to import."
```

```
12        pass_with_warnings: "This delivery is importable, but please review the flagged
rallies above."
13        fail: "This delivery has blocking issues that must be fixed before it can be imported
? see 'Things to review'."
14
15      sections:
16        - { id: run,        title: "Run",               kind: identity,   audience: both }
17        - { id: video,      title: "Video",             kind: video_meta, audience: both }
18        - { id: summary,    title: "Rally summary",     kind: highlights, audience: customer,
item_noun: rally }
19        - { id: review,     title: "Things to review",  kind: flags,      audience: customer
}
20        - { id: stages,     title: "Processing stages", kind: stages,     audience: technical
}
21        - { id: findings,   title: "Findings & issues", kind: flags,      audience: technical
}
22        - { id: verdict,    title: "Verdict",           kind: verdict,    audience: both }
23
24      rules:
25        # A non-commercial license means the video is kept but EXCLUDED from the default
26        # (commercial-only) export ? easy to forget when it later "goes missing" downstream.
27        - code: license_noncommercial
28          severity: warn
29          when:
30            - { path: "stages.registration.details.is_commercial", op: eq, value: false }
31          message: >-
32            This video's license is non-commercial, so it is EXCLUDED from the default
33            commercial-only export. It stays in the DB; pass --include-noncommercial to
34            export it.
35          faq_ref: "docs/INGESTION_PIPELINE.md#will-a-non-commercial-license-block-export"
36
37        # No fps recorded at import -> downstream export/manifest carries no fps and the
38        # indexer must probe the file; provide --fps for accurate frame<->second timing.
39        - code: missing_fps
40          severity: warn
41          when:
42            - { path: "video_meta.fps", op: eq, value: null }
43            - { path: "stages.registration.metrics.segments_registered", op: is_present }
44          message: >-
45            No fps was recorded for this video. The export manifest will lack fps and the
46            indexer will have to probe the file. Pass --fps for accurate timing.
47          faq_ref: "docs/INGESTION_PIPELINE.md#how-are-timestamps-handled-the-fps-thing"
48
```

12. assistant (2026-06-07T08:16:08.475Z)

This spec.yaml is the collector's spec, not the indexer's. The report under evaluation is the badminton-highlight-indexer report. The FAQ referenced is "README#Q5". Let me find the indexer's spec and README/FAQ, since the report says `see README#Q5`. Let me search the harness eval fixtures directory which is where this report came from.

13. user (2026-06-07T08:16:09.666Z)

```
total 820
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:45 .
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:46 ..
-rw-r--r-- 1 avidu 197609 4224 Jun  7 13:40 collector_fixtures.json
-rw-r--r-- 1 avidu 197609 1062 Jun  7 13:41 combine.py
-rw-r--r-- 1 avidu 197609 3508 Jun  7 13:40 gen_collector.py
-rw-r--r-- 1 avidu 197609 4595 Jun  7 13:40 gen_indexer.py
-rw-r--r-- 1 avidu 197609 7154 Jun  7 13:40 indexer_fixtures.json
-rw-r--r-- 1 avidu 197609 3810 Jun  7 13:41 manifest.json
-rw-r--r-- 1 avidu 197609 3812 Jun  7 13:41 manifest_stdout.txt
-rw-r--r-- 1 avidu 197609 2008 Jun  7 13:45 render_html.py
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:41 reports
```

```
---reports---
total 36
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:41 .
drwxr-xr-x 1 avidu 197609    0 Jun  7 13:45 ..
-rw-r--r-- 1 avidu 197609  976 Jun  7 13:41 collector_missed_rally.report.md
-rw-r--r-- 1 avidu 197609 1117 Jun  7 13:41 collector_noncommercial.report.md
-rw-r--r-- 1 avidu 197609  922 Jun  7 13:41 collector_overlap.report.md
-rw-r--r-- 1 avidu 197609 1339 Jun  7 13:41 indexer_fps_fallback.report.md
-rw-r--r-- 1 avidu 197609 1422 Jun  7 13:41 indexer_motion_synthetic.report.md
-rw-r--r-- 1 avidu 197609 1464 Jun  7 13:41 indexer_no_api_key.report.md
-rw-r--r-- 1 avidu 197609 1453 Jun  7 13:41 indexer_validation_failed.report.md
```

14. user (2026-06-07T08:16:11.900Z)

```
1       """Generate broken-run indexer reports (technical markdown) for the debuggability
eval."""
2       import json, os, sys
3       sys.path.insert(0, os.getcwd())  # indexer repo root
4       import obsreport
5       from backend.reporting import harvest
6       from backend.validators.base import ValidationResult
7
8       OUT = r"C:\Users\avidu\AppData\Local\Temp\obseval\indexer_fixtures.json"
9
10      FFPROBE_OK = {
11          "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001",
"codec_name": "hevc"}],
12          "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
13      }
14
15      def vr(name, passed, msg="ok"):
16          return ValidationResult(validator_name=name, passed=passed, message=msg)
17
18      def tech(env, spec):
19          return obsreport.render_technical(env, spec)
20
21      scenarios = []
22
23      # S1: no API key + 0 segments -> silent_provider_noop
24      os.environ.pop("GEMINI_API_KEY", None)
25      rec = harvest.record_run(
26          video_id="md5a", video_path="/videos/match.mp4",
27          config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
28          val_results=[vr("format_check", True), vr("relevance_check", True)],
val_context={"ffprobe_meta": FFPROBE_OK},
29          play_segments=[], candidates=[{"id": 1}],
30          segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
was_reingest=False, segmentation_ran=True)
31      scenarios.append({"id": "indexer_no_api_key", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
32                        "truth": "The run produced 0 highlight segments because no AI-provider
API key was set, so the AI segmenter silently no-op'd. It is NOT that the video has no rallies.
Fix: set GEMINI_API_KEY (e.g. in a .env file) and re-run.",
33                        "expected_faq": "Q7"})
34
35      # S2: motion segmenter -> synthetic_motion_metadata
36      os.environ["GEMINI_API_KEY"] = "x"
37      rec = harvest.record_run(
38          video_id="md5b", video_path="/videos/match.mp4",
39          config={"indexing": {"default_segmenter": "motion"}},
40          val_results=[vr("format_check", True), vr("relevance_check", True)],
val_context={"ffprobe_meta": FFPROBE_OK},
41          play_segments=[{"duration": 5.0, "metadata": {"detector": "motion"}}],
candidates=[],
```

```
42          segmenter_requested="motion", segmenter_actual="motion", was_reingest=False,
segmentation_ran=True)
43      scenarios.append({"id": "indexer_motion_synthetic", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
44                        "truth": "The 'motion' segmenter was used; its per-segment
metadata/events are heuristic/synthetic, not measured, so they must not be trusted as ground
truth. Use yolo_hybrid or gemini for real shot metadata.",
45                        "expected_faq": "Q10"})
46
47      # S3: validation failed (blur)
48      rec = harvest.record_run(
49          video_id="md5c", video_path="/videos/match.mp4",
50          config={"indexing": {"default_segmenter": "wasb_hybrid"}, "ai_provider": "gemini"},
51          val_results=[vr("format_check", True), vr("blur_check", False, "Focus check failed.
Average sharpness 0.0 below threshold 20.0."), vr("relevance_check", True)],
52          val_context={"ffprobe_meta": FFPROBE_OK}, play_segments=[], candidates=[],
53          segmenter_requested=None, segmenter_actual=None, was_reingest=False,
segmentation_ran=False)
54      scenarios.append({"id": "indexer_validation_failed", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
55                        "truth": "Indexing was halted because a validation check failed: the
blur/focus check (video too blurry / sharpness below the min threshold). Fix the footage quality
or lower validation.min_blur_threshold in config.json, then re-run.",
56                        "expected_faq": "Q13"})
57
58      # S4: fps fallback (no ffprobe meta but segmented)
59      rec = harvest.record_run(
60          video_id="md5d", video_path="/videos/match.mp4",
61          config={"indexing": {"default_segmenter": "wasb_hybrid"}, "ai_provider": "gemini"},
62          val_results=[vr("format_check", True)], val_context={},
63          play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}],
64          segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
was_reingest=False, segmentation_ran=True)
65      scenarios.append({"id": "indexer_fps_fallback", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
66                        "truth": "fps was not probed from the file (a fallback default was
used), so motion thresholds and frame->second timing may be miscalibrated. Confirm the real fps
with ffprobe and ensure validation ran.",
67                        "expected_faq": "Q5"})
68
69      with open(OUT, "w", encoding="utf-8") as f:
70          json.dump(scenarios, f, indent=2)
71      print(f"wrote {len(scenarios)} indexer scenarios -> {OUT}")
72
```

15. assistant (2026-06-07T08:16:16.697Z)

Now I understand the setup. The indexer repo is a sibling. The report references README#Q5 in the indexer repo. Let me find the indexer's README and its Q5 to evaluate whether the FAQ reference matches the FPS fallback scenario, since the reader flagged a potential Q5/keyframes mismatch.

16. user (2026-06-07T08:16:17.789Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.pytest_cache/README.md
/c/Users/avidu/Projects/badminton-highlight-indexer/README.md
---
Annotation Setup
CODE WITH ERIC
Email-Mining
KhelSutraGuru.jpeg
Verifiers
antigravity_analysis_results.md
badminton-highlight-indexer
```

```
badminton-highlight-indexer-khelacharya
gtest
muneem
rally-annotator
sports-data-collector
sports-obsreport
```

17. user (2026-06-07T08:16:21.908Z)

```
14:    *   *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
36:The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
50:*   Validate `ffmpeg` and `ffprobe` are present in your PATH.
114:        "min_fps": 29.9,
158:*   `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30 fps
? 10 samples/sec.
168:### Q1: The server complains that `ffprobe` is not found.
169:Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py`
outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
187:### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
188:Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard
FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs.
We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer
level instead, which is incredibly robust and executes instantly without decoding a single
pixel.
197:### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
242:### Q10: Can I trust the per-segment metadata from the `motion` segmenter?
256:### Q13: A validation check failed and indexing was halted ? what do I do?
257:[Omitted long matching line]
288:The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
293:*   `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
316:  utils/                  # ffprobe/ffmpeg + bbox geometry helpers
```

18. assistant (2026-06-07T08:16:26.812Z)

Now I can see the critical issue. Q5 is titled **"Why did the video fail with 'No keyframes / packet timestamps could be extracted'?"** ? it's about keyframe/packet extraction failure in HEVC streams, NOT about fps fallback. Let me read the full Q5 and Q1 to confirm the mismatch, and check whether there's a more appropriate FAQ for fps.

19. user (2026-06-07T08:16:27.578Z)

```
166     ## ? Frequently Asked Questions (FAQ)
167
168     ### Q1: The server complains that `ffprobe` is not found.
169     Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py`
outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
170
171     ### Q2: Why does my video fail the Relevance check?
172     Our default relevance check scans for the typical indoor green badminton court floor
color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust
the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and
`court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
173
174     ### Q3: How do I add a new custom validator to the pipeline?
175     1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
176     2. Subclass `VideoValidator` and implement `validate(self, video_path, config,
context)`.
177     3. Register your class inside `backend/validators/__init__.py` using
```

`registry.register(MyCheckValidator())`.
    It's that simple! The app will automatically run it, display it in the CLI diagnostics summary, and render it in the frontend checklist.

    ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
    *   **Print Buffering**: Standard Python buffers console output when stdout is redirected or running in background shells on Windows. To get real-time unbuffered log statements immediately, run commands with Python's unbuffered flag:
        ```bash
        python -u -m backend.main --video-path "C:/path/to/video.mp4"
        ```
    *   **Video Seeking Overhead**: Seeking to random frames in high-bitrate long H.265/HEVC videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The `motion` segmenter reads sequentially and skips frame decodes using container-level `cap.grab()`, which delivers a **100x speedup** on multi-gigabyte raw files. `yolo_hybrid` sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.

    ### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
    Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer level instead, which is incredibly robust and executes instantly without decoding a single pixel.

    ### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
    You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first N sub-sampled frames, enabling you to test motion filters and court relevance checks in seconds:
    ```bash
    python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300
    ```
    (At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60 seconds of video).

    ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
    Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:
    1. Create a local `.env` file in the project root (this is already in `.gitignore` to prevent secret leaks).
    2. Set your Google AI Studio API key in the file:
       ```env
       GEMINI_API_KEY=your_key_here
       ```
    3. Run with whichever segmenter you want:
       ```bash
       # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid

       # Full-video Gemini analysis (more expensive, highest semantic precision per call)
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini

       # Pure CV baseline ? no API key needed
       python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
       ```
       Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under `"indexing"` in `config.json`.

    The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` + `ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers can be added under `backend/providers/`.

    ### Q8: How does the `yolo_hybrid` pipeline actually save money?

Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs in four phases:

1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`, `chunk_overlap`) so we can process incrementally.
2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-`min_action_window_duration` windows discarded.
3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to `ai_max_workers`.
4. **DB population** ? confirmed segments are written to `play_segments` along with `shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.

Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?

Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:

1. Open your `config.json` file.
2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):

```json
"indexing": {
    "default_segmenter": "gemini",
    "debug_duration": 15.0,
    ...
}
```

3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

Q10: Can I trust the per-segment metadata from the `motion` segmenter?

No. The zero-dependency `motion` segmenter detects *timing* via pixel-difference, but its per-segment **metadata is heuristic/synthetic** ? `intensity`, `avg_speed_kmh`, the serve/smash/foul/winner **events**, and `server`/`receiver` are generated values, **not measured**. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use `yolo_hybrid` (default) or `gemini`. If your observability report shows a `synthetic_motion_metadata` warning, this is why.

Q11: I re-ran a video and my previous results changed/disappeared. Why?

Videos are keyed by the **MD5 of the file's bytes**. Re-processing the *same* file is treated as a fresh ingest: `add_video` is `INSERT OR REPLACE`, and the foreign-key cascade **deletes the previous play/candidate segments** before writing the new ones. So the earlier results are intentionally replaced, not merged. (A `reingest_replaced_prior` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

Q12: What is the `*.report.json` / `*.report.md` / `*.summary.md` file next to my video?

Every processing run now writes a **per-video observability report** next to the video (or to `report.output_dir` if set in `config.json`):

- **`<name>.report.json`** ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.
- **`<name>.report.md`** ? the **technical** report: validation results, processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ entry to

read. **Start here when a run looks wrong.**
252   - **`<name>.summary.md`** ? a **customer-friendly** summary: just the video metadata and highlights, no system internals.
253
254   To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a `? see README#Qn` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."}` in `config.json`.
255
256   ### Q13: A validation check failed and indexing was halted ? what do I do?
257   Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_*` in `config.json`); **format** (not an MP4 container); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.
258
259   ---
260
261   ## ? Annotation & Evaluation
262
263   Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.
264
265   ```bash

20. assistant (2026-06-07T08:16:32.243Z)

This confirms the core issue. Now let me verify what message the report actually generated and which FAQ ref the spec rule attaches. The report cites `README#Q5`, and Q5 is specifically titled about keyframes/packet timestamps extraction failure ? a different problem from fps fallback. Let me find the indexer's spec rule for `fps_fallback_used` to confirm the source of the `faq_ref: Q5`.

21. user (2026-06-07T08:16:33.399Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:29:
"""Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:30:
distinguishes a real probe from the absence of one (the fallback footgun)."""
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:31:    meta:
Dict[str, Any] = {"fps_source": "unknown"}
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:52:
meta["fps_source"] = "probed"
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:182:    # used
its internal fallback (30fps / 1280px) ? surface that as the footgun it is.
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:183:    if
vm.get("fps_source") == "unknown" and segmentation_ran:
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/harvest.py:184:
vm["fps_source"] = "fallback"
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:56:  - code:
fps_fallback_used
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:59:      - {
path: "video_meta.fps_source", op: eq, value: fallback }
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:61:      fps was
assumed (a fallback default), not probed from the file. Motion
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:64:    faq_ref:
"README#Q5"
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:87:
explicit --segmenter was given ? so a fallback default kicked in. The indexer's
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/reporting/spec.yaml:113:      reason
(relevance -> README#Q2, fps/keyframes -> README#Q5).
```

```
Binary file /c/Users/avidu/Projects/badminton-highlight-
indexer/backend/reporting/__pycache__/harvest.cpython-313.pyc matches
```

22. assistant (2026-06-07T08:16:36.287Z)

Let me read the full fps_fallback rule in the indexer spec to see the exact message and FAQ ref.

23. user (2026-06-07T08:16:36.798Z)

```
54
55        # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
56        - code: fps_fallback_used
57          severity: warn
58          when:
59            - { path: "video_meta.fps_source", op: eq, value: fallback }
60          message: >-
61            fps was assumed (a fallback default), not probed from the file. Motion
62            thresholds and frame->second timing may be miscalibrated. Confirm the real
63            fps with `ffprobe` and ensure validation ran (it probes the file).
64          faq_ref: "README#Q5"
65
66        # The motion baseline fabricates its per-segment metadata/events ? they are not
67        # measured. Warn so nobody trusts them as ground truth.
68        - code: synthetic_motion_metadata
69          severity: warn
70          when:
71            - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
72          message: >-
73            The 'motion' segmenter was used. Its per-segment metadata (intensity, speed,
```

24. assistant (2026-06-07T08:16:58.280Z)

I now have a complete picture. Let me assess.

Ground truth: fps was not probed (fallback default used), so motion thresholds and frame->second timing may be miscalibrated. Confirm real fps with ffprobe and ensure validation ran. Expected FAQ: Q5.

Reader's diagnosis: Correct and precise. They identified fps fallback, the miscalibration consequence, cited the `fps_fallback_used` warning and the `FPS | ? (fallback)` row, and prescribed ffprobe + re-run + ensuring validation runs. This nails the root cause.

FAQ: They consulted README#Q5, which is the expected FAQ. So `faq_correct` is true by the literal expectation.

The gap: The reader themselves flagged the most important real defect ? Q5 is titled *"Why did the video fail with 'No keyframes / packet timestamps could be extracted'?"* and its body is entirely about HEVC/GDR/CRA keyframe extraction at the demuxer level. It says nothing about fps fallback, fps probing, or timing miscalibration. So while the reader landed on the *correct cause* (because the warning message itself is self-explanatory and doesn't depend on the FAQ), the FAQ pointer is **mistargeted**: Q5 does not explain or resolve the fps-fallback issue. The reader was led to conflate "fps fallback" with "HEVC keyframe failure" and even suggested adding a Q1 (ffprobe not found) cross-reference. The correct cause was recovered *despite* Q5, not *because* of it.

This is a real spec/FAQ defect: the `fps_fallback_used` rule points at a FAQ that doesn't cover fps fallback. The reader's actionable fix (re-run so fps probes; ensure validation ran) came from the warning message, not Q5. A reader strictly relying on the cited FAQ would be misdirected toward HEVC keyframe debugging.

Other valid confusion the reader raised:
- The fallback numeric value isn't shown (`FPS | ? (fallback)` rather than e.g. `30 (fallback)`) ? legitimate; harvest.py comment even notes the internal fallback is "30fps / 1280px", so the value is known and could be surfaced.

- Validation passed (1/1) yet fps fell back ? the linkage is genuinely confusing. The message says "ensure validation ran (it probes the file)" but the report shows validation ran and passed, which seems contradictory. In the fixture, only `format_check` ran (no fps-probing validator), so 1/1 passing is technically consistent, but the report doesn't make that visible.

So: cause correct, FAQ consulted-as-expected, but a **minor-to-major** gap because the cited FAQ is genuinely wrong for this scenario. I'll rate it **minor** since the reader still self-debugged correctly and reached the right fix ? but the single highest-value refinement is fixing the FAQ target.

The most concrete, highest-leverage refinement is to point the `fps_fallback_used` rule at a FAQ that actually addresses fps, not keyframes ? and add such an FAQ. Let me propose precisely.

25. user (2026-06-07T08:17:04.615Z)

Structured output provided successfully

26. assistant (2026-06-07T08:17:12.658Z)

I've completed the evaluation and submitted my structured assessment.

Summary: The fresh reader landed on the **correct root cause** (fps fallback causing miscalibrated motion thresholds and frame->second timing) and prescribed the right fix (confirm fps with ffprobe, re-run so it probes, ensure validation ran). They also consulted the literally-expected FAQ (Q5).

However, the real defect ? which the reader themselves caught ? is that **Q5 is mistargeted**. In the indexer README, Q5 is titled "Why did the video fail with 'No keyframes / packet timestamps could be extracted?'" and its entire body is about HEVC/GDR/CRA demuxer keyframe extraction. It says nothing about fps probing or fps fallback. The reader recovered the cause from the self-explanatory `fps_fallback_used` warning message, not from the FAQ ? and a reader who trusted the cited FAQ would be misdirected toward HEVC keyframe debugging (the reader even suggested cross-referencing Q1 ffprobe-not-found as a result).

My single highest-leverage refinement: repoint `spec.yaml:64` from `README#Q5` to a new fps-specific FAQ entry that (a) names the numeric fallback default (harvest.py documents it as 30fps/1280px, yet the report hides it as `FPS | ? (fallback)`), (b) explains why validation can show 1/1 passed while fps still fell back (no fps-probing validator ran), and (c) gives the ffprobe + re-run fix.

Rated the gap **minor** ? correct self-debug was achieved, but the FAQ pointer would actively misdirect anyone relying on it.